PRACTICAL QUESTION PAPER

MongoDB • Express.js • Node.js

Subject	Backend Development	**Duration**	3 Hours
Domain	Library Management System	**Maximum Marks**	100

General Instructions

1. Build a Node.js application using Express.js and MongoDB.
2. Use Mongoose for database connectivity, schema definition, validation, and CRUD operations.
3. Use JSON request bodies and **res.json()** responses for CRUD APIs. Test endpoints using **Thunder Client**.
4. Create the required general pages in both unstyled and professionally styled versions, as specified.
5. Use meaningful variable names, suitable HTTP methods and status codes, and a clear folder structure.

Domain: Library Management System

Develop an application to manage library books. Each book record must contain the following five fields:

Field	Data Type	Validation
title	String	Required; minimum 3 characters
author	String	Required
isbn	String	Required; unique
category	String	Required
price	Number	Required; minimum 1

Question 1: Project Setup and Database Connectivity — 15 Marks

a. Create a Node.js project and install the required dependencies. **(3 marks)**

b. Configure an Express server and define the required middleware. **(3 marks)**

c. Connect the application to MongoDB using Mongoose. **(4 marks)**

d. Store the MongoDB connection string in an environment variable using a `.env` file. **(2 marks)**

e. Display a suitable success or failure message for database connectivity. **(3 marks)**

Question 2: Mongoose Schema and Validation — 15 Marks

a. Define the Book schema and create the Mongoose model using the five fields listed above. **(5 marks)**

b. Implement all specified schema validations. **(5 marks)**

c. Return appropriate validation errors when invalid data is submitted. **(3 marks)**

d. Prevent duplicate ISBN values and handle duplicate-key errors. **(2 marks)**

Question 3: CRUD Operations — 25 Marks

Implement complete CRUD functionality for the Library Management System using Node.js, Express.js, MongoDB, and Mongoose. Test all API endpoints using **Thunder Client**.

Create: Create a POST API to add a new book to the database. Return the response using `res.json()`. **(5 marks)**

Read: Create GET APIs to retrieve all books and fetch an individual book by its MongoDB ID. Return the results using `res.json()`. **(5 marks)**

Update: Create a PUT API to update an existing book using its MongoDB ID. Return the updated record using `res.json()`. **(5 marks)**

Delete: Create a DELETE API to delete a book using its MongoDB ID. Return an appropriate JSON confirmation message. **(5 marks)**

Additional requirements: Validate incoming request data, handle invalid MongoDB IDs, manage errors such as books not found, and return appropriate JSON success or failure messages with suitable HTTP status codes. **(5 marks)**

> **Important: Use JSON request bodies and `res.json()` responses for all CRUD operations. Test every endpoint using Thunder Client. Do not use EJS or frontend forms for this question.**

Question 4: Page Creation and UI Development — 25 Marks

Create the following four pages in both an unstyled version and a professionally styled version, wherever specified. These pages are separate from the JSON CRUD API testing in Question 3.

A. Home page — 7 marks

Display the application name, a welcome message, navigation links, and a list of available books. Create unstyled and professionally styled versions.

B. About page — 4 marks

Describe the purpose of the library system and include information about its features. Implement styled and unstyled versions.

C. Contact page — 5 marks

Provide name, email, and message fields; validate submitted information and display an appropriate response. Implement styled and unstyled versions.

D. Error page — 9 marks

Create a custom 404 page for undefined routes; handle server errors; provide a link back to the home page; implement styled and unstyled error pages; do not expose sensitive technical information.

UI expectations: Without UI means basic HTML elements without CSS frameworks. With UI means Bootstrap 5 or custom CSS, responsive layouts, consistent navigation, clear spacing, styled tables/forms, and alerts.

Question 5: Express Routing, Middleware and Application Structure — 20 Marks

a. Create separate route files for book operations and general pages. **(5 marks)**

b. Implement middleware to parse form data and JSON requests. **(3 marks)**

c. Use reusable navigation and footer components or partials where applicable. **(4 marks)**

d. Implement centralized error-handling middleware. **(4 marks)**

e. Organize the application into suitable folders, such as `models`, `routes`, `views`, `public`, and `middleware`. **(4 marks)**

Final Submission Checklist

■ MongoDB connection working

■ Mongoose schema and validation completed

■ Create, Read, Update, and Delete APIs working and tested in Thunder Client

■ Home page styled and unstyled

■ About page styled and unstyled

■ Contact page styled and unstyled

■ Custom 404 and server error pages

■ Express routes and middleware implemented

■ Project runs without unhandled errors

Total: 100 Marks

Expected outcome: A working Library Management System using Node.js, Express.js, MongoDB, and Mongoose, with validated JSON CRUD APIs tested through Thunder Client, general pages, and robust error handling.